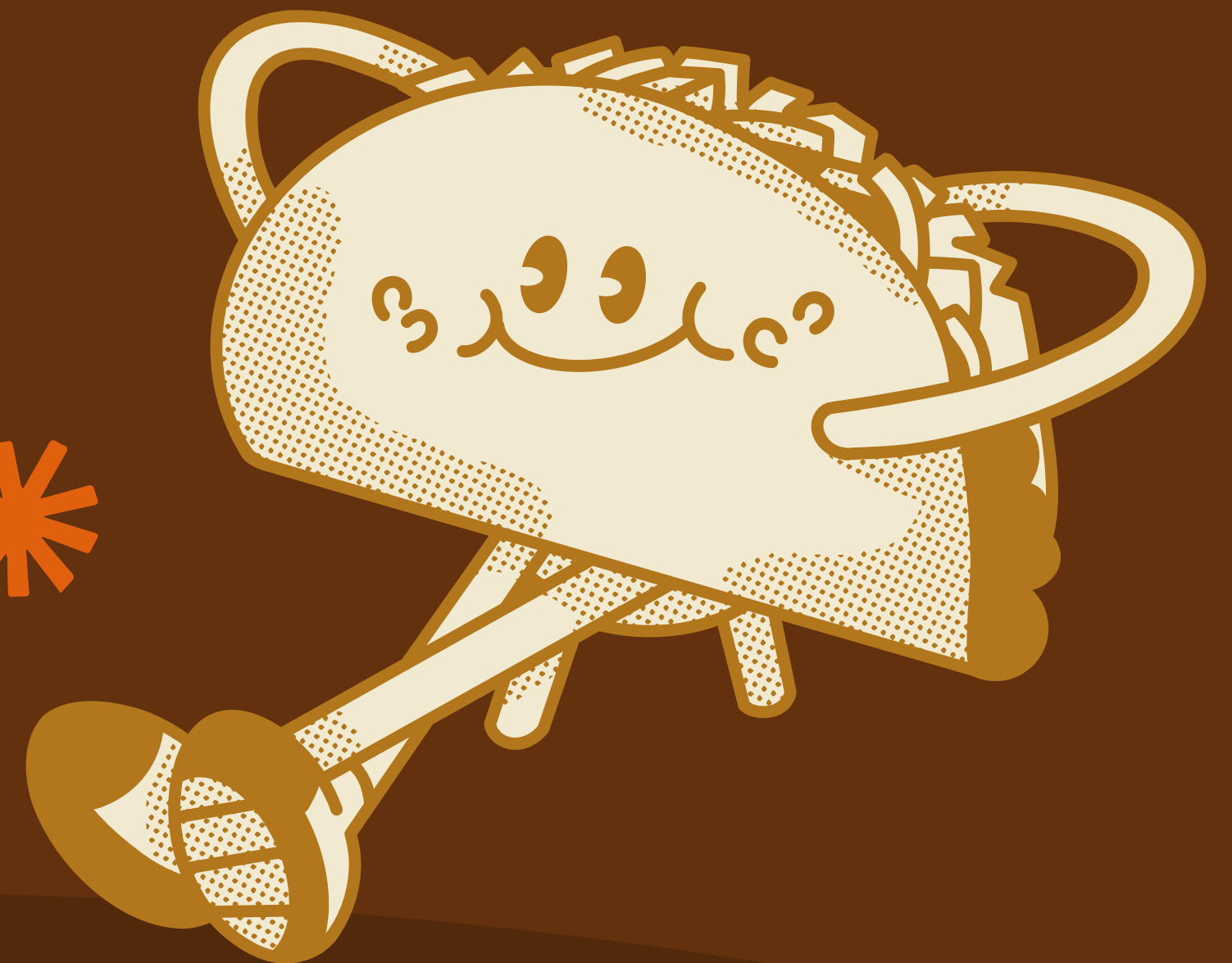
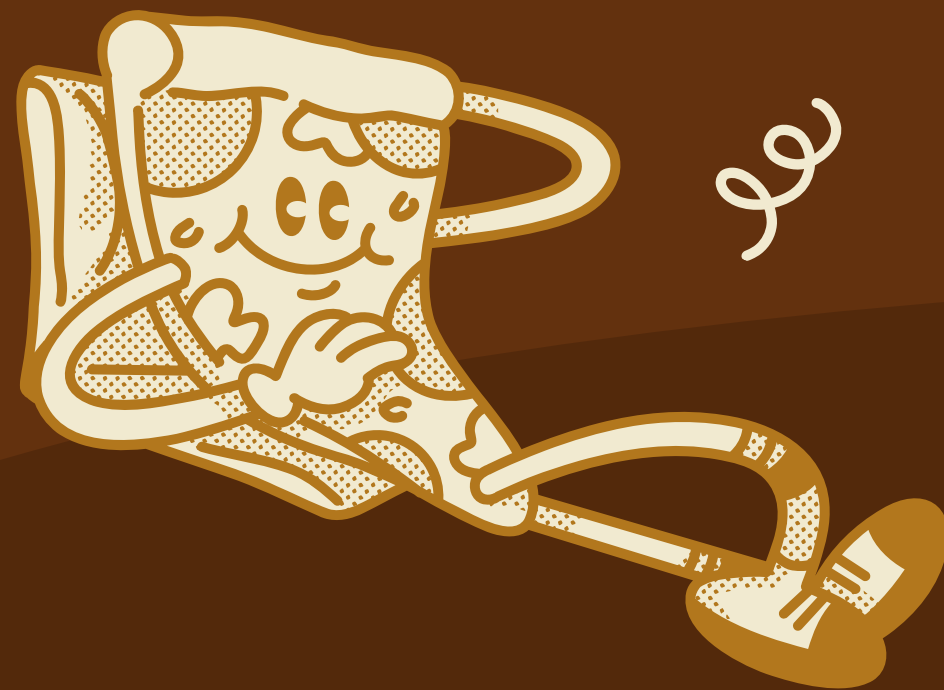
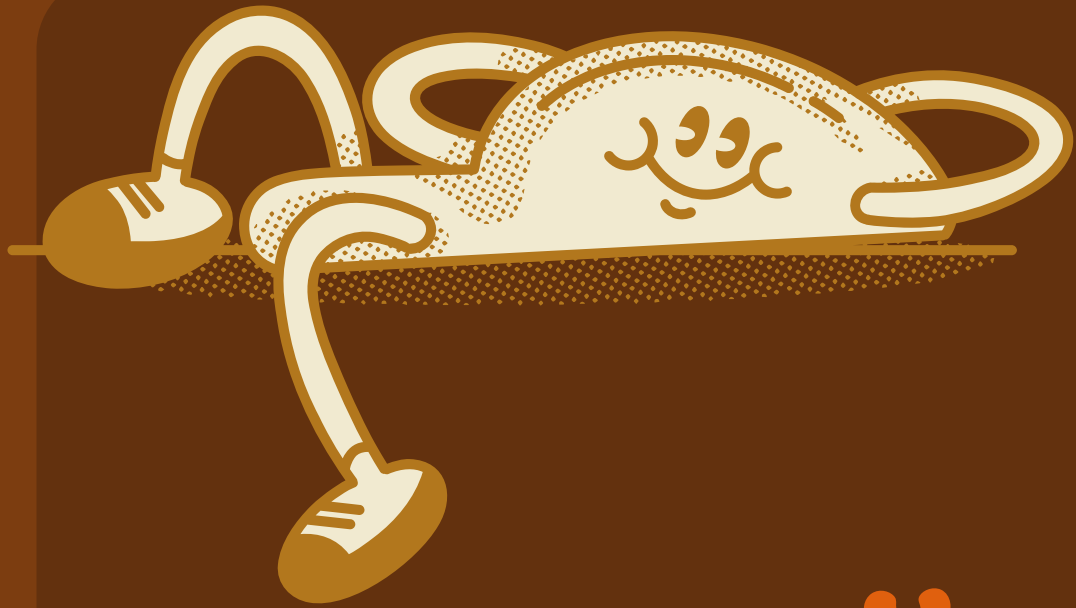


BROSTER PIZZASALES..... PERFORMANCE *



SQL Insights for BROSTER pizza



Hey everyone!

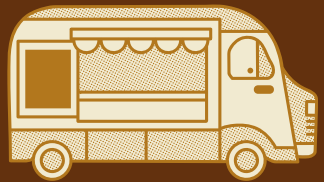


My name is Ashish kumar, my focus is on analyzing pizza order data to discover patterns in customer choices, product success, and business efficiency.



DATASET:

The In this project, I worked with four primary relational tables: Orders, Order_Details, Pizzas, and Pizza_Types.



Model View



☰ pizza_types	...
Abc category	
Abc ingredients	
Abc name	
🔑 pizza_type_id	×
Collapse ^	



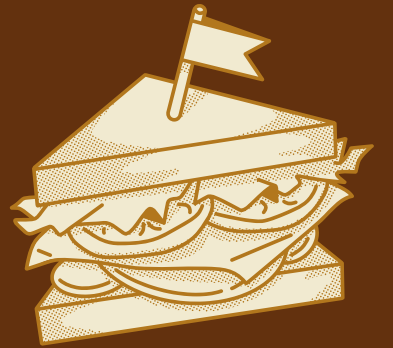
🕒 orders	...
📅 date	
🔑 order_id	×
🕒 time	
Collapse ^	

🍕 pizzas	...
🔑 pizza_id	×
🔑 pizza_type_id	×
Σ price	
Abc size	
Collapse ^	

Σ order_details	...
Σ order_details_id	
🔑 order_id	×
🔑 pizza_id	×
Σ quantity	
Collapse ^	

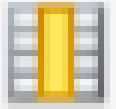


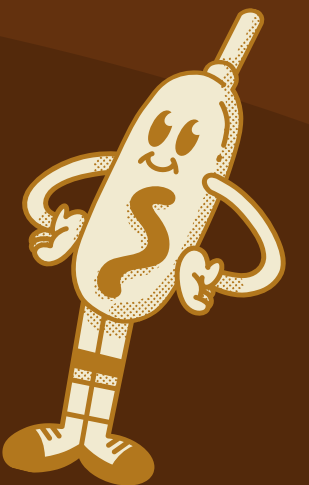
1) Retrieve the total number of orders placed.



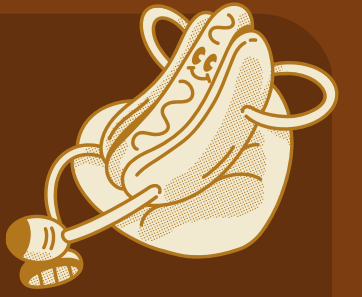
```
SELECT
    COUNT(order_id) AS Total_orders
FROM
    orders;
```

output :

Result Grid 	
	Total_orders
▶	16382



2) Identify the highest-priced pizza

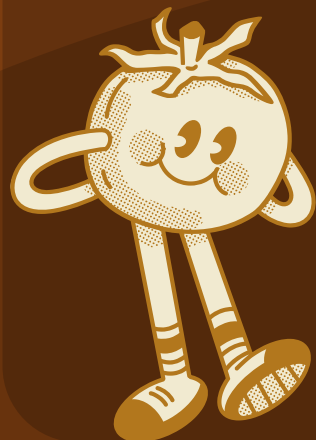


```
SELECT
    pt.name, p.price
FROM
    pizzas p
    JOIN
    pizza_types pt ON pt.pizza_type_id = p.pizza_type_id
WHERE
    p.price = (SELECT
                MAX(price) AS MAX_PRICED
            FROM
                pizzas);
```



Output:

Result Grid			Filter Rows:
	name	price	
▶	The Greek Pizza	35.95	



3) Identify the most common pizza size ordered.



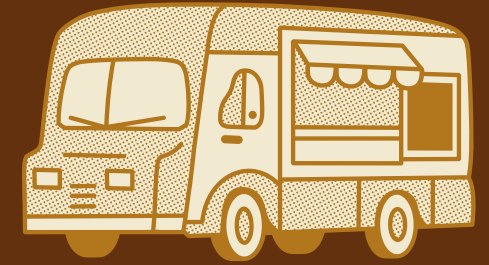
```
SELECT
    p.size, COUNT(od.order_details_id) AS order_count
FROM
    pizzas p
    JOIN
    order_details od ON p.pizza_id = od.pizza_id
GROUP BY p.size
ORDER BY order_count DESC;
```



Output:

Result Grid			Filter Rows:
	size	order_count	
▶	L	950	
	M	775	
	S	676	
	XL	30	

4) List the top 5 most ordered pizza types along with their quantities

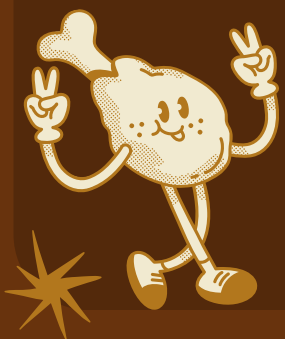


```
SELECT
    pizza_types.name, SUM(order_details.quantity) AS quantity
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
    JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY quantity DESC
LIMIT 5;
```



Output:

Result Grid   Filter Rows:		
	name	quantity
▶	The Pepperoni Pizza	153
	The Barbecue Chicken Pizza	125
	The Thai Chicken Pizza	118
	The California Chicken Pizza	116
	The Sicilian Pizza	114

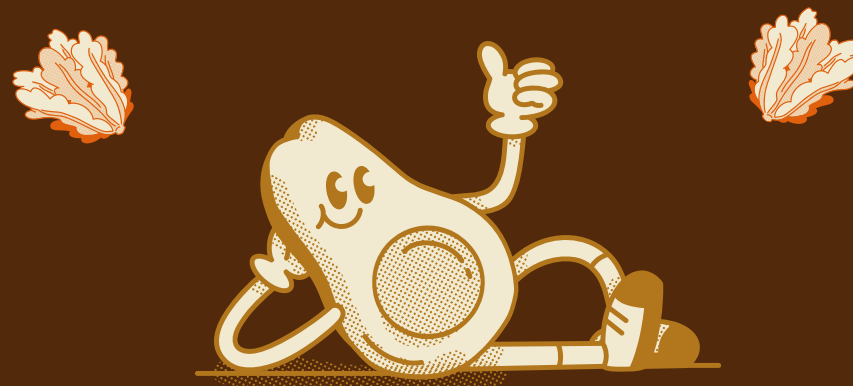


5) Calculate the total revenue generated from pizza sales.

Output:

```
SELECT
    ROUND(SUM(od.quantity * p.price), 2) AS Total_sales
FROM order_details od
    JOIN pizzas p
    ON p.pizza_id = od.pizza_id;
```

Result Grid	
	Total_sales
▶	40999.35



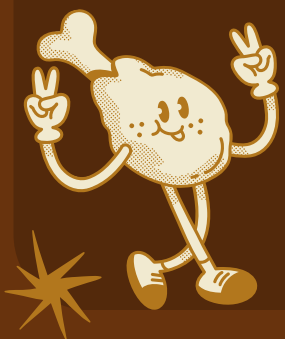
6) Join the necessary tables to find the total quantity of each pizza category ordered. ✨

```
select pt.category,  
sum(od.quantity) as quantity  
from pizza_types pt  
join pizzas p  
on pt.pizza_type_id = p.pizza_type_id  
join order_details od  
on od.pizza_id = p.pizza_id  
group by pt.category order by quantity desc limit 5;
```

Output:



	category	quantity
▶	Classic	730
	Supreme	619
	Veggie	586
	Chicken	546

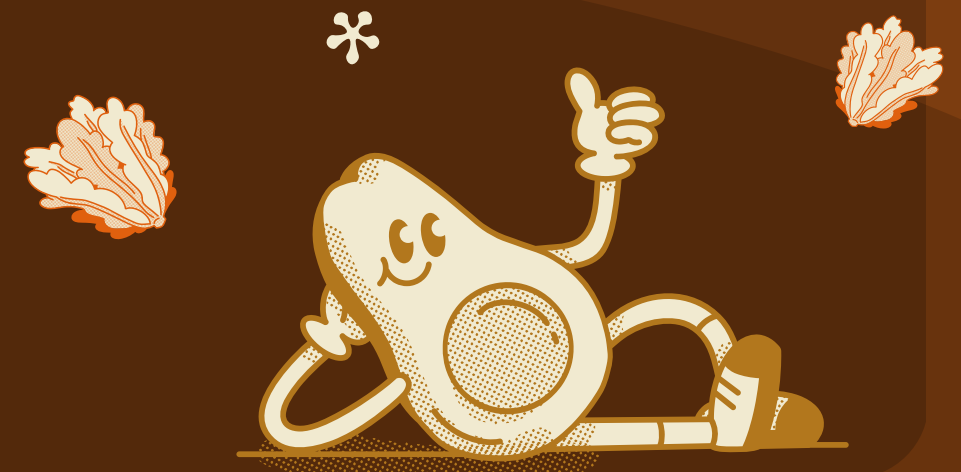


7) Determine the distribution of orders by hour of the day.

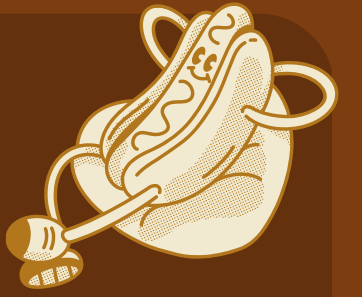
Output:

Result Grid	Filter Rows:
HOURS	order_Count
9	19176
10	73999
11	9698060
12	16868787
13	17134240
14	11627459
15	11894806
16	14478782
17	15691642
18	16614264
19	15155954
20	12664653
21	10443943
22	6227758
23	300836

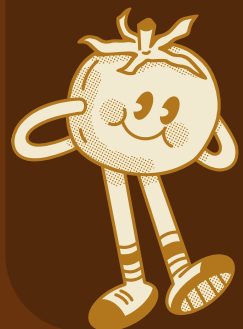
```
SELECT
    HOUR(order_time) AS HOURS, SUM(order_id) AS order_Count
FROM
    orders
GROUP BY HOUR(order_time);
```



8) Group the orders by date and calculate the average number of pizzas ordered per day.



```
SELECT
    ROUND(AVG(quantity), 0) as Avg_pizza_ordered_per_day
FROM
    (SELECT
        o.order_date, SUM(od.quantity) AS quantity
    FROM
        orders o
    JOIN order_details od ON o.order_id = od.order_id
    GROUP BY o.order_date) AS order_quantity;
```



Output:

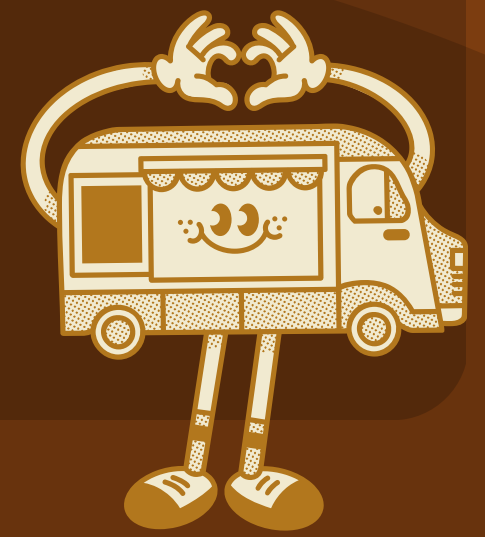
Result Grid		Filter Rows:
	Avg_pizza_ordered_per_day	
▶	129	

9) Determine the top 3 most ordered pizza types based on revenue.

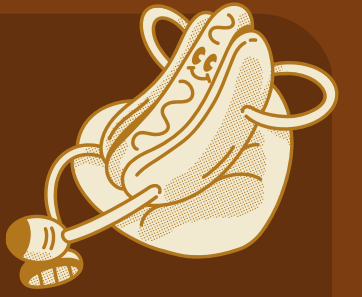
Output:

```
SELECT
    pt.name, SUM(od.quantity * p.price) AS Revenue
FROM
    pizza_types pt
    JOIN
    pizzas p ON pt.pizza_type_id = p.pizza_type_id
    JOIN
    order_details od ON p.pizza_id = od.pizza_id
GROUP BY pt.name
ORDER BY Revenue DESC
LIMIT 3;
```

Result Grid			Filter Rows:
	name	Revenue	
▶	The Barbecue Chicken Pizza	2225.75	
	The Thai Chicken Pizza	2188.5	
	The California Chicken Pizza	2003	



10) Join relevant tables to find the category-wise distribution of pizzas.

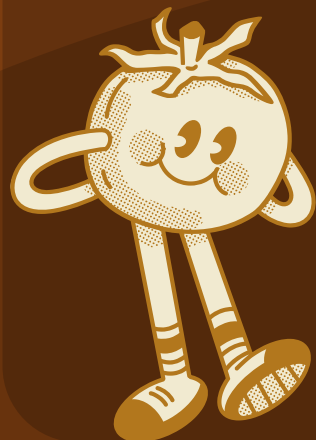


```
select category, count(name) from pizza_types  
group by category;
```



Output:

Result Grid			Filter Rows:
	category	count(name)	
▶	Chicken	6	
	Classic	8	
	Supreme	9	
	Veggie	9	

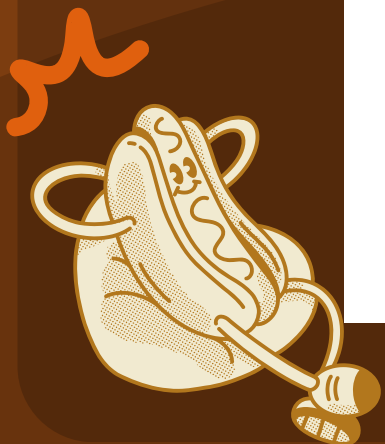


11) Calculate the percentage contribution of each pizza type to total revenue.

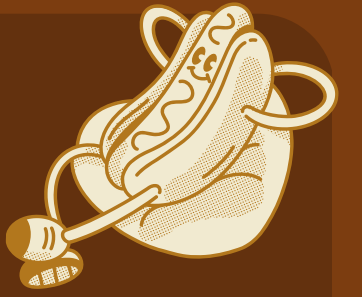
```
SELECT
  pt.category,
  ROUND(SUM(od.quantity * p.price) * 100 / (SELECT
    SUM(od2.quantity * p2.price)
  FROM
    order_details od2
    JOIN
    pizzas p2 ON p2.pizza_id = od2.pizza_id),
  2) AS REVENUE_percentage
FROM
  pizza_types pt
  JOIN
  pizzas p ON pt.pizza_type_id = p.pizza_type_id
  JOIN
  order_details od ON od.pizza_id = p.pizza_id
GROUP BY pt.category
ORDER BY REVENUE_percentage DESC;
```

Output:

Result Grid			Filter Rows:
	category	REVENUE_percentage	
▶	Classic	26.24	
	Supreme	25.94	
	Veggie	24.07	
	Chicken	23.76	



12) Analyze the cumulative revenue generated over time.

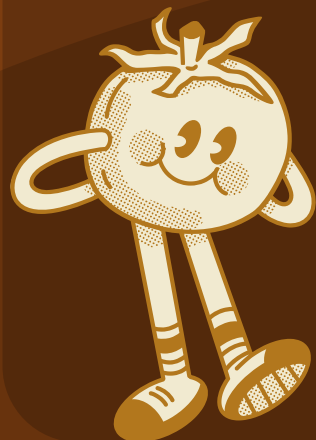


```
select o.order_date,  
round(  
sum(  
sum(od.quantity * p.price)) over (order by o.order_date),2)as cumulative_revenue  
from order_details od  
join pizzas p  
on od.pizza_id = p.pizza_id  
join orders o  
on o.order_id = od.order_id  
group by o.order_date  
order by order_date asc;
```



Result Grid  Filter Rows:		
	order_date	cumulative_revenue
▶	2015-01-01	2713.85
	2015-01-02	5445.75
	2015-01-03	8108.15
	2015-01-04	9863.6
	2015-01-05	11904.8
	2015-01-06	14333.75
	2015-01-07	16535.95
	2015-01-08	19374.3
	2015-01-09	21476.9
	2015-01-10	23924.1
	2015-01-11	25796.4
	2015-01-12	27715.45
	2015-01-13	29752.55
	2015-01-14	32279.95
	2015-01-15	34236
	2015-01-16	36638.2
	2015-01-17	38646.05

Output:



13) Determine the top 3 most ordered pizza types based on revenue for each pizza category.



```
select name, revenue from
(
  select category, name, revenue,
  rank() over(partition by category order by revenue desc) as ranks
  from
    (select pt.category, pt.name,
    sum(od.quantity * p.price) as revenue
    from pizza_types pt
    join pizzas p
    on pt.pizza_type_id = p.pizza_type_id
    join order_details od
    on od.pizza_id = p.pizza_id
    group by pt.category, pt.name) as a) as b
where ranks <= 3;
```

Output:

Result Grid	Filter Rows:
name	revenue
The Barbecue Chicken Pizza	2225.75
The Thai Chicken Pizza	2188.5
The California Chicken Pizza	2003
The Pepperoni Pizza	1948.25
The Classic Deluxe Pizza	1784.5
The Greek Pizza	1469
The Italian Supreme Pizza	1874
The Spicy Italian Pizza	1808
The Sicilian Pizza	1760.5
The Five Cheese Pizza	1480
The Four Cheese Pizza	1462.4...
The Mexicana Pizza	1379.5



*Thank
you!*